

VIJAY
PANDEY



Spring Boot — Made Simple 

Spring Boot

Request Flow

What happens *after* you hit an API?



No Jargon • Just Clarity



Request Entry

Client → Controller

Why it matters: This is where every request enters your application. A clean entry point makes APIs easier to manage and debug.

Action:

- Accept HTTP requests here
- Map URLs clearly
- Avoid business logic in controllers

Example:

A user sends GET **/users**.

UserController receives the request.



Business Logic

Controller → Service

Why it matters: The service layer keeps your business logic separate from request handling, making code reusable and testable.

Action:

- Move all business logic into services
- Keep controllers thin
- Reuse services across multiple controllers

Example:

A controller calls **`userService.getAllUsers()`** instead of handling logic itself.



Data Access

Service → Repository

Why it matters: Repositories isolate database logic, so your application doesn't depend directly on how data is stored.

Action:

- Use repositories only for database operations
- Avoid writing SQL in services
- Let Spring Data JPA handle queries

Example:

The service calls **`userRepository.findAll()`** to fetch data from the database.



Database Layer

Repository → Database

Why it matters: This layer ensures data is stored and retrieved efficiently and securely.

Action:

- Use entities to map database tables
- Let Spring manage transactions
- Keep database logic centralized

Example:

The repository fetches user records from the database using JPA and returns them to the service.



Response Handling

Back to Client

Why it matters: Spring Boot automatically converts Java objects into JSON responses, saving time and reducing boilerplate.

Action:

- Return objects, not JSON manually
- Let Spring handle serialization
- Keep response logic simple

Example:

The repository fetches user records from the database using JPA and returns them to the service.



Final Takeaway

End-to-End

Why it matters: Understanding the request flow helps you debug faster, scale applications, and write cleaner code.

Action:

- Think in layers, not files
- Debug step by step
- Respect separation of concerns

Example:

When an API fails, you instantly know whether the issue is in the controller, service, or repository.

